Tema 7
Arquitectura y diseño de software

Arquitecturas de software

PDS - Procesos de Desarrollo de Software 2025/26

Contenidos

1. Motivación.
2. Patrones arquitecturales.
3. Ejercicios.

1. Motivación

¿Qué es y para qué sirven las arquitecturas de software?

1. Motivación

La arquitectura es el diseño de alto nivel.

Pensamos en cómo se organiza **TODO** el sistema.

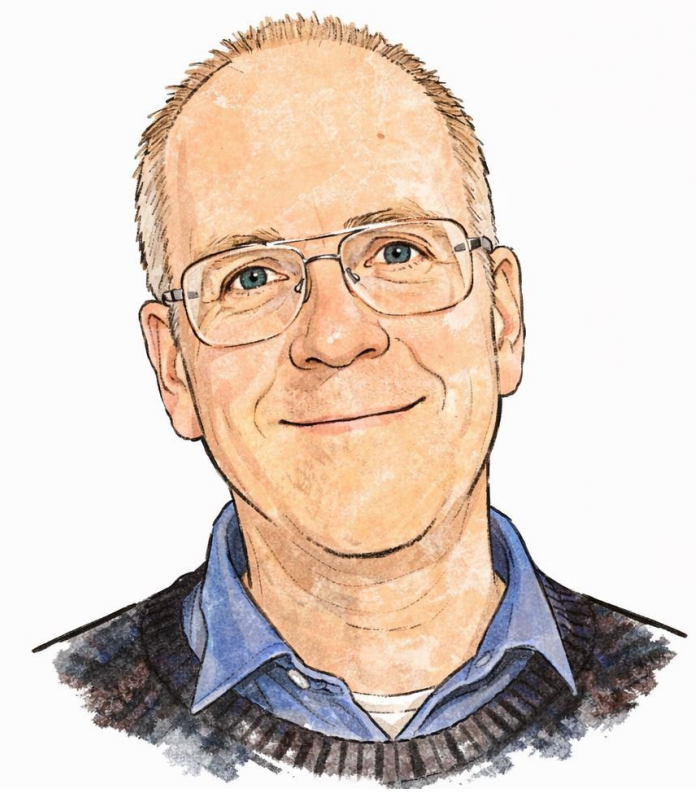
- Dejamos de pensar en unidades pequeñas *(clases, objetos, métodos, …)*
- Ahora pensamos en: paquetes, módulos, componentes, subsistemas, capas, servicios, …

Objetivos:

- Representar las partes principales del software.
- Definir cómo se relacionan.
- Permitir que el sistema evolucione.

1. Motivación – ¿Qué es lo importante?

“**Architecture** is about the important stuff. Whatever that is.”



Ralph Johnson
(Coautor de "Design Patterns", GoF)

1. Motivación – ¿Qué es lo importante?

¿Qué es *lo importante*?

Lo importante es lo que es **difícil de cambiar**.

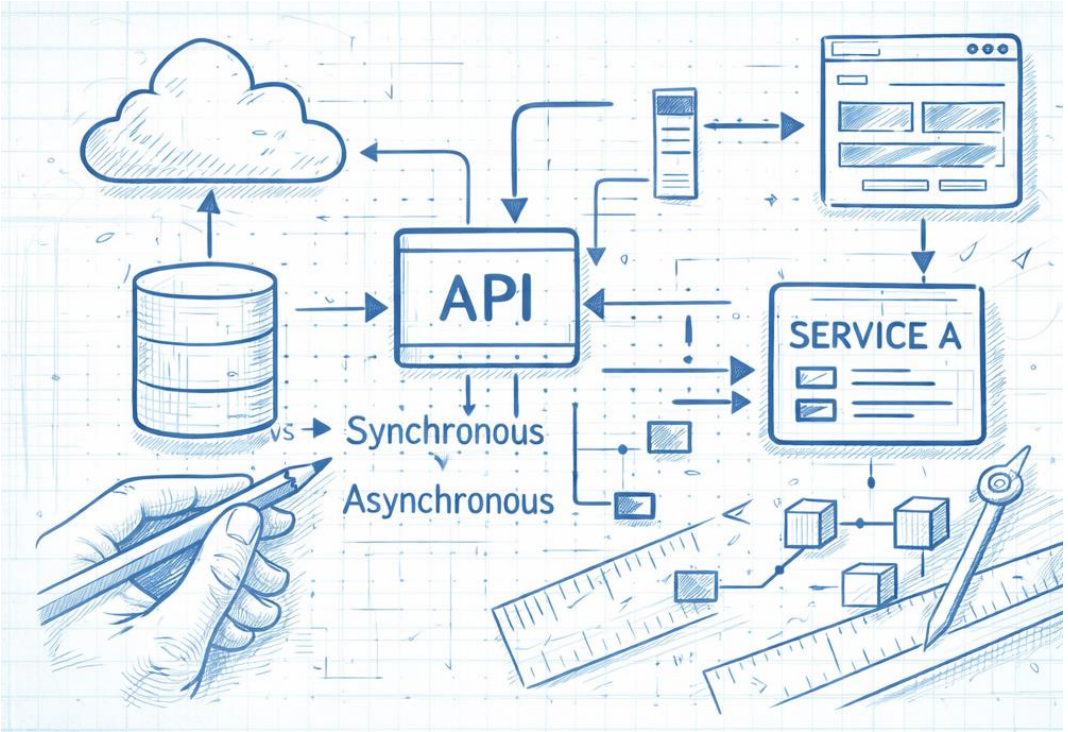
- **Cómo se organiza el sistema.**
  No como se implementa.
- **Cómo se comunican sus partes.**
  Y que no se acoplen sus módulos.
- **Decisiones que afectan a todo y que queramos acertar desde el principio.**
  No cosas que podamos arreglar después.

> "**Architecture** is about the important stuff. Whatever that is."

Ralph Johnson
(Coautor de "Design Patterns", GoF)

1. Motivación – Definición

- Permite razonar a **alto nivel**.
- Formada por elementos *(componentes)* y las relaciones entre ellos.
- Pero sobre todo, está formada por las **decisiones principales del sistema**.
- Son los **planos del software**: estructura, comportamiento, interacciones, propiedades no funcionales, …



¿Qué decisión es más importante?

- ¿Calculamos el total del carrito con un bucle o un **stream**?
- ¿Usamos un `Map` o un `ArrayList` para guardar los elementos del carrito?
- ¿La comunicación con el sistema de pagos será síncrona o asíncrona?

1. Motivación – Rol de la arquitectura

Problemas

- Todo en el mismo bloque de código.
- No hay funciones para modularizar.
- Variables globales desordenadamente.
- No hay reutilización de código.
- **¡Pero funciona!**

¿Por qué no escribir el código así?

- ¿Cómo lo podemos mejorar?
- ¿Con qué objetivo?

```python
productos = []
precios = []
while True:
  prod = input("Ingrese producto (o 'fin'):")
  if prod == 'fin':
    break
  productos.append(prod)
  precio = float(input("Ingrese precio: "))
  precios.append(precio)

suma = 0
for p in precios:
  suma += p
impuesto = suma * 0.15
total = suma + impuesto

print("Resumen de compra:")
for i in range(len(productos)):
  print(f"{productos[i]} - ${precios[i]:.2f}")
print(f"Subtotal: ${suma:.2f}")
print(f"Impuesto (15%): ${impuesto:.2f}")
print(f"Total a pagar: ${total:.2f}")
```

1. Motivación – Rol de la arquitectura

¿Por qué no escribir el código así?

Porque es **difícil de cambiar**.

- Si se quiere modificar impuestos u otra funcionalidad, es difícil hacerlo.
- Al hacer un cambio, no sabemos de manera clara cuál va a ser su impacto.

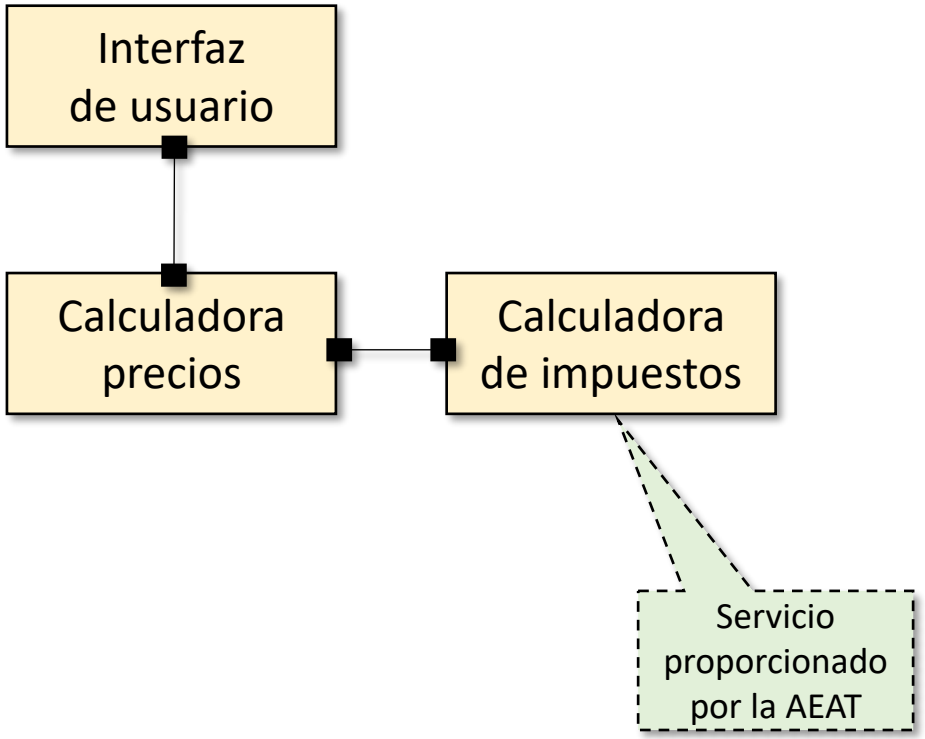
```python
productos = []
precios = []
while True:
  prod = input("Ingrese producto (o 'fin'):")
  if prod == 'fin':
    break
  productos.append(prod)
  precio = float(input("Ingrese precio: "))
  precios.append(precio)

suma = 0
for p in precios:
  suma += p
impuesto = suma * 0.15
total = suma + impuesto

print("Resumen de compra:")
for i in range(len(productos)):
  print(f"{productos[i]} - ${precios[i]:.2f}")
print(f"Subtotal: ${suma:.2f}")
print(f"Impuesto (15%): ${impuesto:.2f}")
print(f"Total a pagar: ${total:.2f}")
```

1. Motivación – Rol de la arquitectura

- La arquitectura juega el mismo rol que el diseño modular de software
- La granularidad es a nivel de sistema.
  - Subsistemas.
  - Servicios.



* Este es un ejemplo forzado, la arquitectura tiene sentido para sistemas más grande

1. Motivación – Rol de la arquitectura

La arquitectura del software tiene como objetivo abordar los **requisitos no funcionales**.

- La arquitectura del software trata con los **factores de calidad** del software
- Ejemplos: *extensible, tolerante a fallos, etc.*

¿Por qué?

- Los requisitos funcionales normalmente se pueden abordar agregando, modificando o eliminando código.
- Los requisitos no funcionales son más costosos de añadir después.
- Los requisitos no funcionales suelen ser *cross-cutting concerns*

1. Motivación – Utilidad

- Nos ayuda a razonar sobre el sistema (responder preguntas)

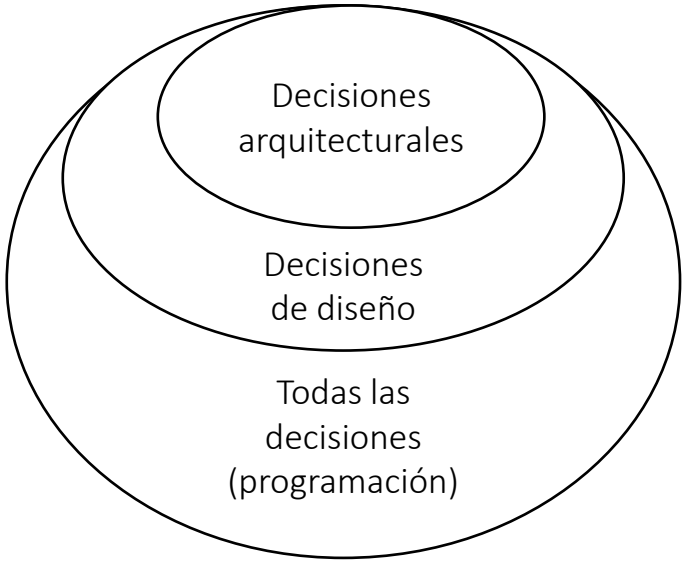
¿Cuáles son las partes del sistema que pueden cambiar?

- ¿Realizar este cambio *romperá* (o afectará) a otra parte del sistema?
- ¿Dónde debo incluir cierta funcionalidad?
- ¿Qué significa esta decisión que hay en el código?
  - Las decisiones arquitecturales terminan llegando al código

1. Motivación – Arquitectura vs diseño

- Toda la arquitectura es diseño.
- No todo el diseño es arquitectura.

	Tipo de especificación	Ámbito
Arquitectura	Abstracto	Sistema completo
Diseño	Abstracto	Partes del sistema
Implementación	Concreto	Partes del sistema



* *Architecture, Design, Implementation*, Eden & Kazman, ICSE'93

1. Motivación – Elementos de la arquitectura

Componentes

- Entidad que encapsula un par de la funcionalidad o lo datos del sistema.
- Se comunican a través de **puertos** que tienen una **interfaz** bien definida

Conectores

- Entidad que realiza y regula las interacciones entre los componentes.
- Ejemplos:
  - Llamada a función
  - Memoria compartida
  - Invocación remota (RPC)

2. Estilos arquitectónicos

Diferentes estilos arquitectónicos para diferentes problemas

2. Estilos arquitectónicos – Definición

Un **estilo arquitectónico** caracteriza una familia de sistemas que están relacionados por compartir propiedades estructurales y funcionales.

Un estilo arquitectónico describe un sistema completo.

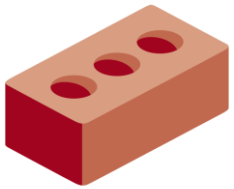
- Una colección de decisiones arquitecturales aplicables a ciertos contextos.
- En contraste, un patrón diseño nos ayuda a diseñar partes de un sistema.

2. Estilos arquitectónicos – Definición

Un estilo arquitectónico responde a las siguientes preguntas:

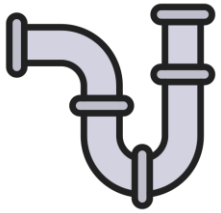
- ¿Cuál es el vocabulario de diseño?
  es decir, los tipos de componentes y conectores.
- ¿Cuáles son los patrones estructurales permitidos?
- ¿Cuál es el modelo computacional subyacente?
- ¿Cuáles son los invariantes esenciales?
- ¿Cuáles son algunos ejemplos comunes de su uso?
- ¿Cuáles son las ventajas y desventajas?

2. Estilos arquitectónicos – Tipos



Arquitectura en capas

Microkernel

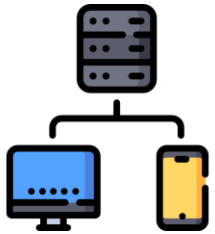


Filtros y tuberías

Basadas en eventos

Publicar y suscribir

Colas de mensajes

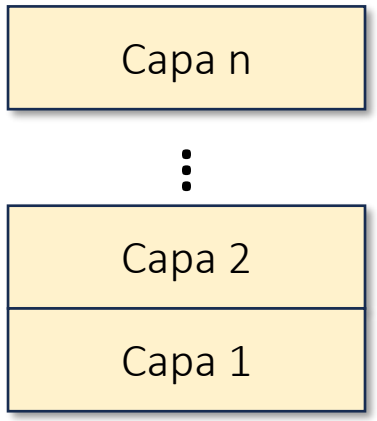


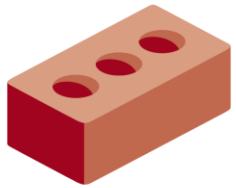
Cliente-servidor

Microservicios

2. Estilos arquitectónicos – Arquitectura en capas

- También conocida como *Layered architecture*
- **Cada capa ofrece una abstracción** de nivel superior.
  - Capas inferiores, más bajo nivel
- **Componentes**
  - Capa, ofrece una API a las capas superiores
  - Dentro de cada capa, los diferentes módulos ofrecen la funcionalidad de la capa
- **Conectores**
  - Llamada a procedimiento (síncrona)
  - Eventos de capas inferiores a superiores (asíncrona)

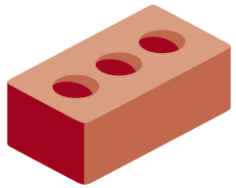




2. Estilos arquitectónicos – Arquitectura en capas

Características

- Cada capa solo tiene una responsabilidad.
  - La capa *k* sólo puede invocar servicios de la capa *k – 1*.
  - La capa *k* no sabe de la existencia de la capa *k + 1*.
- Todos los módulos que forman una capa están al mismo nivel de abstracción entre sí.
- ¿Cómo se puede comunicar una capa inferior con una superior?
  - A través de *callbacks.*
  - La capa superior instala un callback en la inferior.
  - La capa inferior invoca al callback cuando es necesario.



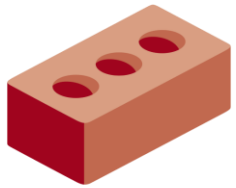
2. Estilos arquitectónicos – Arquitectura en capas

Ventajas

- Reutilización de las capas
- Permite la estandarización (POSIX por ejemplo).
- Las dependencias quedan confinadas (entonces tenemos portabilidad y testeabilidad).
- Intercambiabilidad.

Desventajas

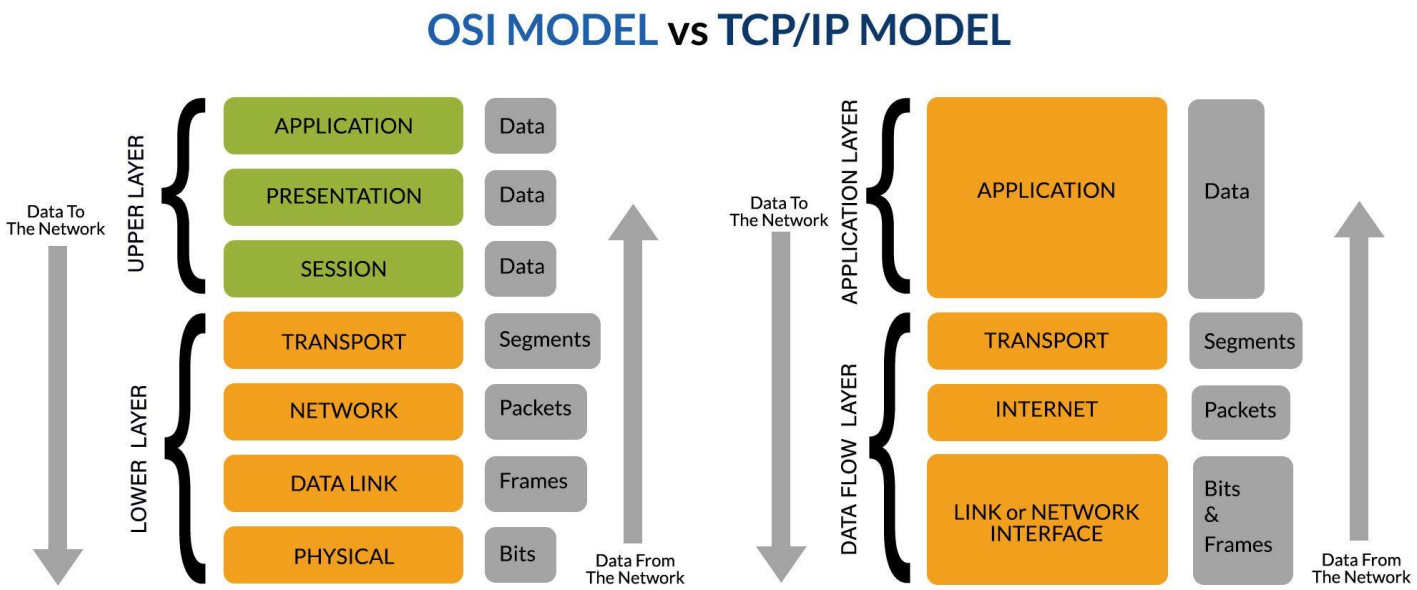
- Problemas si se cambian capas inferiores (no son estables).
- Ineficiencia o cierta duplicación de trabajo cuando hacen falta utilizar servicios de capas inferiores.

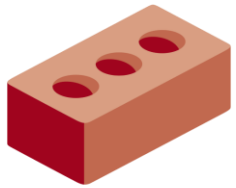


2. Estilos arquitectónicos – Arquitectura en capas

Ejemplos

- Máquinas Virtuales
  (Java Virtual Machine, por ejemplo)
- Sistemas operativos.
- Protocolos de red.
  (por ejemplo, el modelo OSI y o la forma en que se implementan los protocolos TCP/IP).



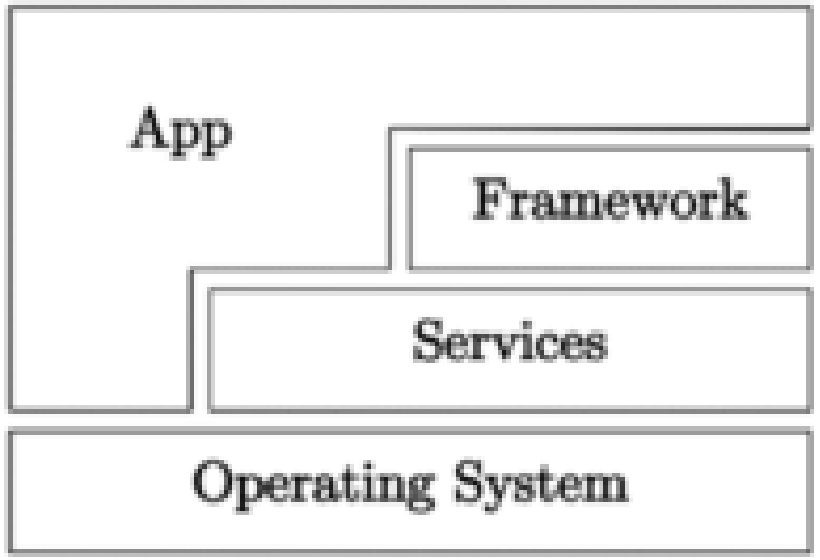


2. Estilos arquitectónicos – Arquitectura en capas

Variantes

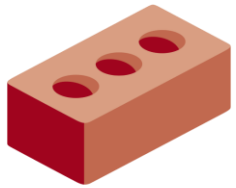
Permitir acceso a ciertos servicios de las capas inferiores.

- Idealmente, cada capa solo accede a la capa inferior.
- Pero en la práctica, a veces nos saltamos capas: Por rendimiento, simplicidad o limitaciones técnicas.

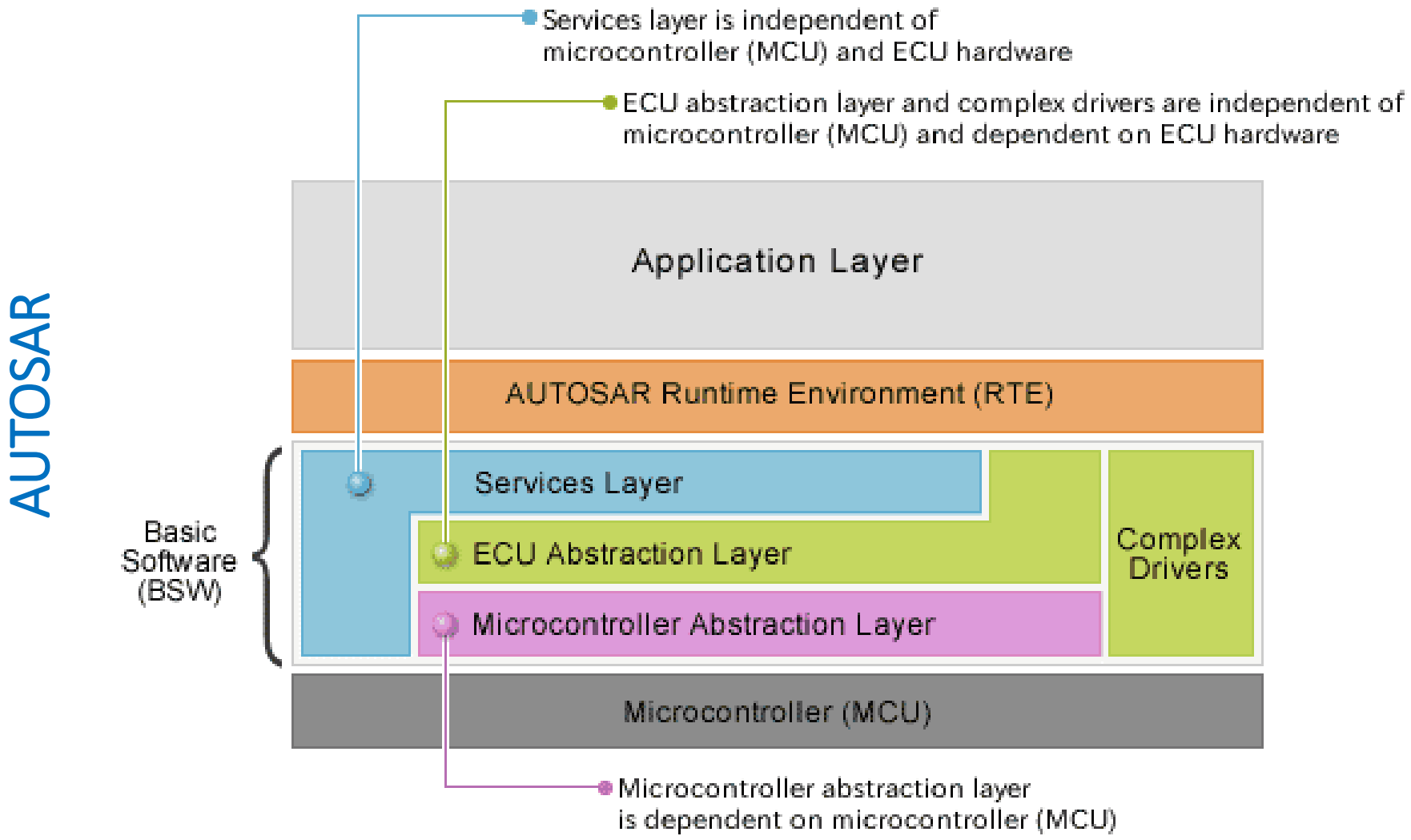


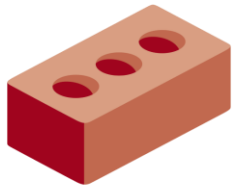
Linux

(invocar a libc o syscalls directamente)



2. Estilos arquitectónicos – Arquitectura en capas

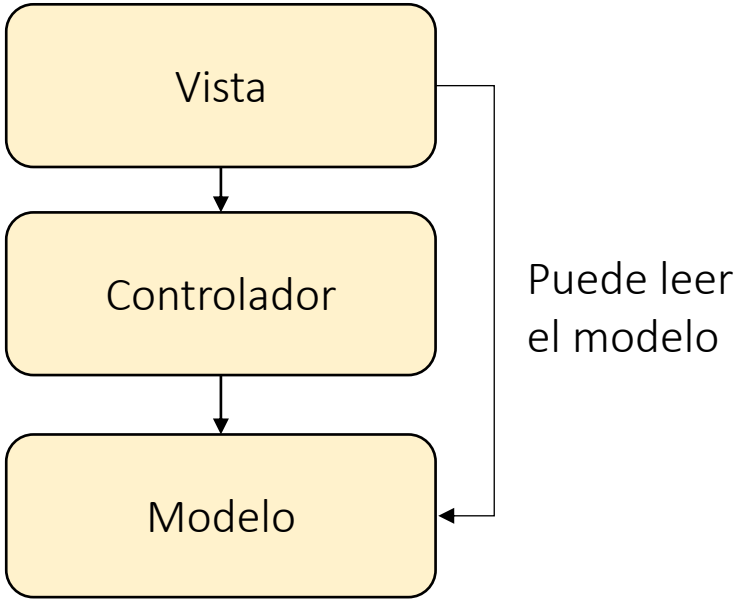




2. Estilos arquitectónicos – Arquitectura en capas

MVC

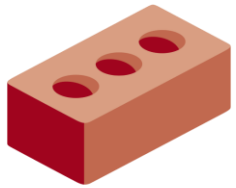
Patrón basado en la arquitectura basada en capas con el objetivo de separar lógica de negocio de la interfaz de usuario.



Componentes:

- **Modelo**: lógica y datos .
- **Vista**: representación.
- **Controlador**: gestiona la interacción .

La vista puede acceder al modelo directamente.

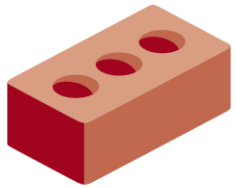


2. Estilos arquitectónicos – Arquitectura en capas

Transaction Script

La lógica de la aplicación se organiza de acuerdo a las transacciones que tiene que realizar el sistema.

- **Un procedimiento independiente** por cada operación de negocio:
  - Valida datos
  - Aplica reglas de negocio
  - Accede a la base de datos
  - Confirma la transacción.
- **Acceso directo a la base de datos** (o con una capa de abstracción sencilla)
- **Útil para para aplicaciones pequeñas con lógica simple**



https://martinfowler.com/eaaCatalog/transactionScript.html

2. Estilos arquitectónicos – Microkernel

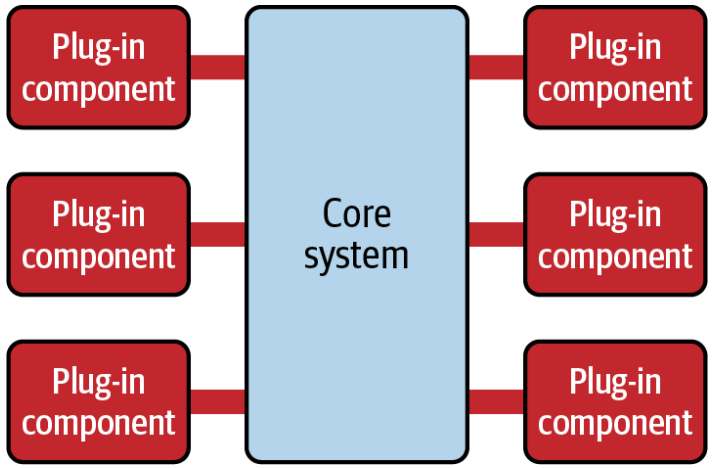
- También conocida *arquitectura basada en plug-ins.*
- Permite añadir funcionalidad a través de extensiones.
  Añadir nueva funcionalidad no requiere modificar el núcleo el sistema

Componentes

- Núcleo del sistema
- Extensiones

Conectores

- En un SO: IPC, memoria compartida, llamadas al sistema, etc.
- En un IDE: APIs de los plugins, eventos, etc.



* https://www.oreilly.com/library/view/software-architecture-patterns/9781098134280/ch04.html

2. Estilos arquitectónicos – Microkernel

Ventajas

- Modular y extensible
  - Sencillo añadir nuevas características.
  - Desarrollo incremental e independiente (varios equipos)
- Fiabilidad y tolerancia a fallos.
  - Si está bien diseñado, el fallo de una extensión no afectará al resto.

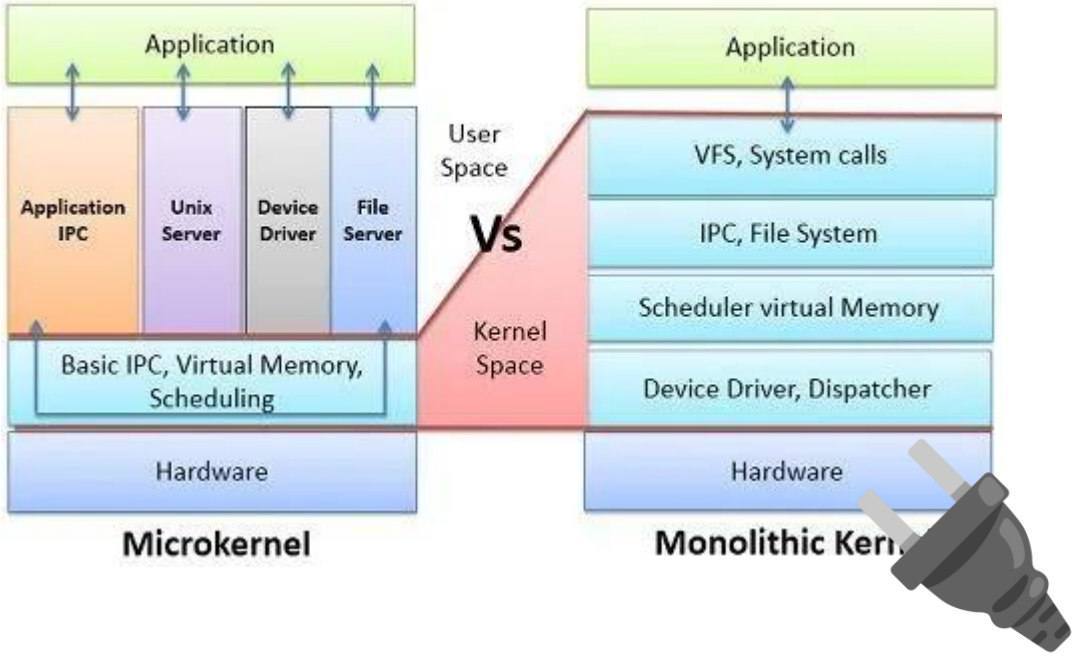
Desventajas

- El núcleo debe ser estable.
  - Si el núcleo cambia, no es posible saber qué extensiones van a romperse
- Problemas de escalabilidad o rendimiento si el núcleo tiene que realizar la coordinación.
- Interacciones indeseadas entre las extensiones.

2. Estilos arquitectónicos – Microkernel

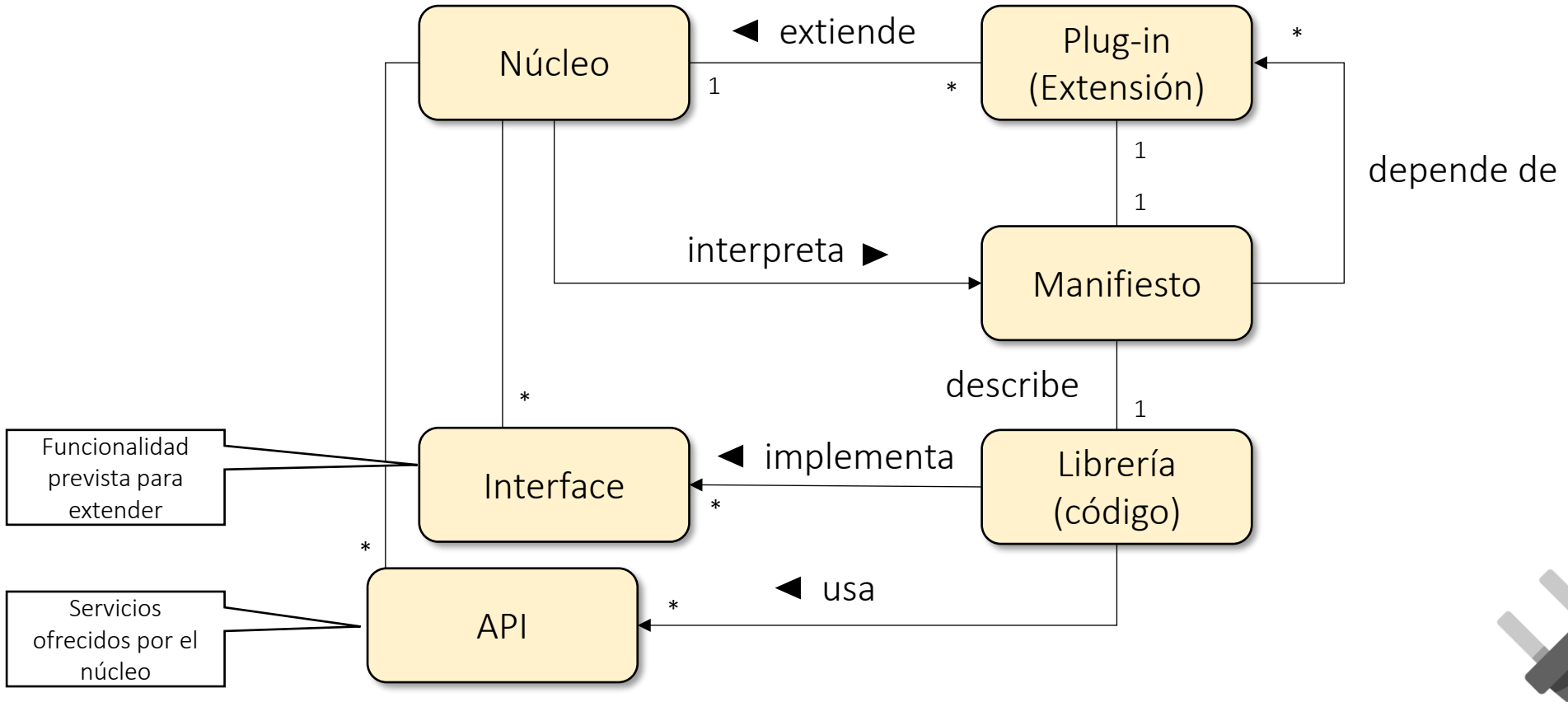
Ejemplos

- Eclipse IDE.
- Visual Studio.
- Jenkins – Servicio integración continua.
- Sistemas operativos.
  - Linux es monolítico.



2. Estilos arquitectónicos – Microkernel

Ejemplo de estructura (*típica en Microkernels OO*)



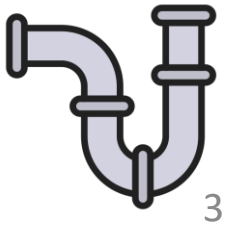
2. Estilos arquitectónicos – Filtros y tuberías

Los **componentes** se denominan *filtros.*

- Se encargan del procesamiento.
- Transforma datos de entrada.
- Genera flujo de salida.

Los **conectores** se denominan *pipe.*

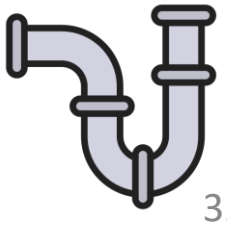
- Conecta un filtro con el entorno (entrada o salida).
- Conecta un filtro con otro filtro.



2. Estilos arquitectónicos – Filtros y tuberías

Características

- Ningún filtro conoce la identidad (ni la existencia) de otros filtros.
- Ningún filtro depende o conoce el estado de los otros filtros.
- La corrección de la salida del sistema no depende del orden en que cada filtro realice sus cálculos.
  - Depende del orden en que se organiza el *pipeline*
- Los tubos no realizan ningún tipo de modificación sobre los datos que transmiten.
  - No hay efectos laterales.



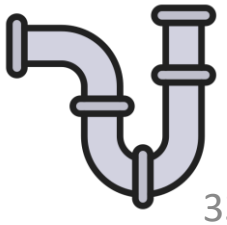
2. Estilos arquitectónicos – Filtros y tuberías

Ventajas

- Composición funcional
  ¿Cuál es la interfaz entre componentes?
- Reutilización
  - Filtros reutilizables en sistemas con el mismo formato de datos.
  - Filtros reemplazables por otros que lean el mismo formato de datos.
- Bajo acoplamiento
  No hay dependencia del contexto

Desventajas

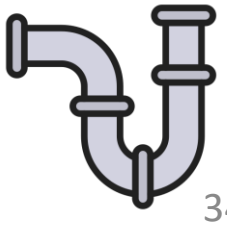
- Dependencia del filtro más lento.
- Dificultad en sistemas interactivos.
- Duplicación del trabajo.
  Copias de los datos



2. Estilos arquitectónicos – Filtros y tuberías

Ejemplos

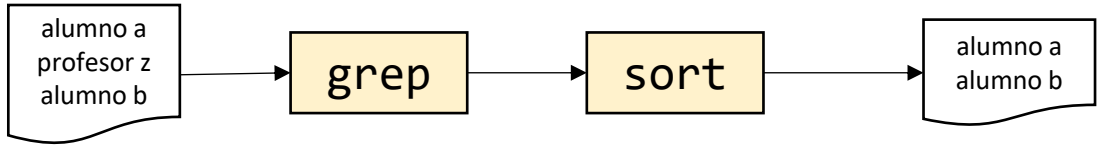
- Procesamiento de señales.
- Procesamiento de imágenes o sonido.
- Compiladores.
- Procesamiento de cadenas.
- Procesamiento de streams (*flujos de datos*).



2. Estilos arquitectónicos – Filtros y tuberías

Ejemplo UNIX

```
cat entrada.txt | grep "alumno" | sort > salida.txt
```



Filosofía UNIX

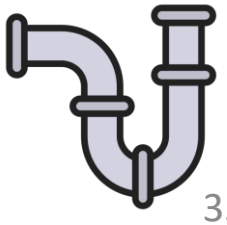
- Programas pequeños.
- Cada uno hace una cosa.
- Se pueden combinar.

Doug McIlroy vs. Donald Knuth

*"Read a file of text, determine the **n** most frequently used words, and print out a sorted list of those words along with their frequencies."*

- Knuth - 10 páginas
  (literate programming)
- McIlroy

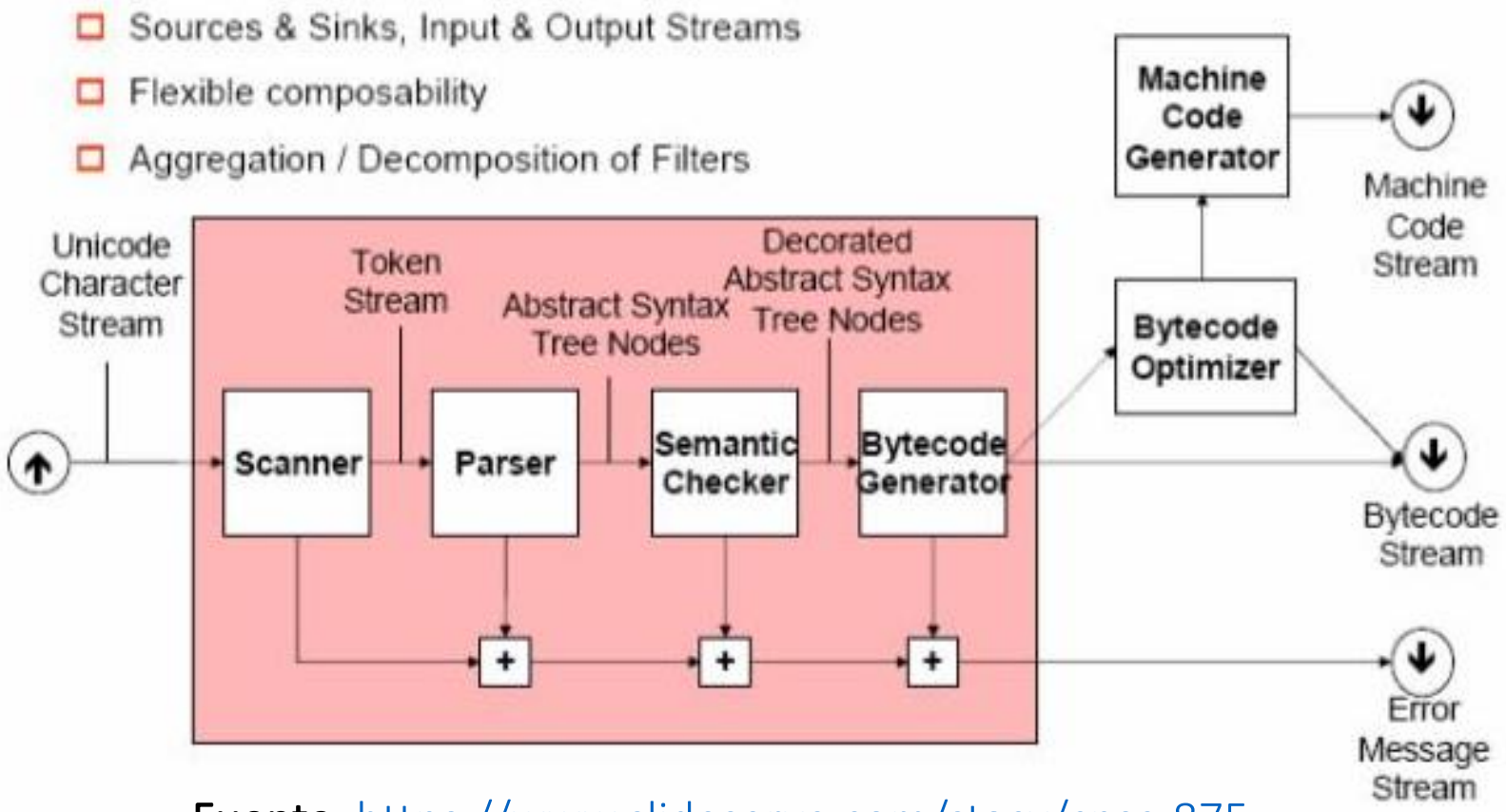
```
tr -cs A-Za-z '\n' |
tr A-Z a-z |
sort |
uniq -c |
sort -rn |
sed ${1}q
```



* http://www.leancrew.com/all-this/2011/12/more-shell-less-egg/

2. Estilos arquitectónicos – Filtros y tuberías

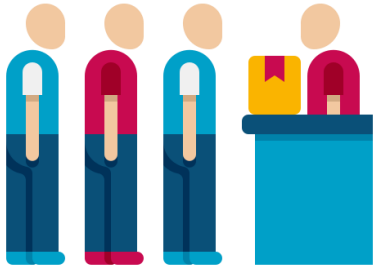
Ejemplo compilador



Fuente: https://www.slideserve.com/stacy/cpsc-875

2. Estilos arquitectónicos – Eventos

- Comunicación desacoplada mediante mensajes.
- Características comunes:
  - No hay llamadas directas entre componentes.
  - Comunicación mediante mensajes/eventos.
  - Emisor y receptor están desacoplados.
- Consideramos dos estrategias.



Colas de mensajes.

Publicar y suscribir.

2. Estilos arquitectónicos – Eventos – Colas de mensajes

Características

- Comunicación mediante mensajes.
- Uso de una cola intermedia.
- Cada mensaje lo procesa un consumidor.
- Comunicación asíncrona.
- Los consumidores leen (modo *pull)* de la cola.
- Los mensajes se procesan en orden.

Roles

- **Productor**: envía mensajes.
- **Consumidor**: procesa mensajes.

2. Estilos arquitectónicos – Eventos – Colas de mensajes

Características

- Comunicación asíncrona.
  Los mensajes se envían y se procesan en momentos diferentes.
- Cola de mensajes.
  Los mensajes se almacenan temporalmente en una cola de mensajes.
- Desacoplamiento.
  Los componentes se comunican sin comunicación directa.
- Entrega fiable de mensajes.
  Los mensajes se almacenan en la cola. No se pierden.
- Cada servicio puede ir a su ritmo.
  Cada servicio pide (*pull*) mensajes cuando tiene capacidad disponible.
- Flujos de trabajo en el que el orden importa.
  Ej: pagar antes de enviar.

2. Estilos arquitectónicos – Eventos – Colas de mensajes

Características - Desacoplamiento

- **Espacial**
  - Los productores y consumidores no se conocen.
  - No es necesario saber quién envía o quién recibirá la información.
- **Temporal**

  Productores y consumidores no necesitan estar disponibles a la vez.
  - Si un consumidor está caído, el productor puede seguir emitiendo mensajes.
  - Cuando el consumidor vuelva, seguirá procesando mensajes.

2. Estilos arquitectónicos – Eventos – Colas de mensajes

Ejemplos

- Integración de aplicaciones empresariales *(e.g., conectar aplicaciones empresariales).*
- Automatización de workflows *(e.g., procesamiento de pedidos, background tasks).*
- Sistemas de mensajes fiables *(e.g., transacciones bancarias, logística).*

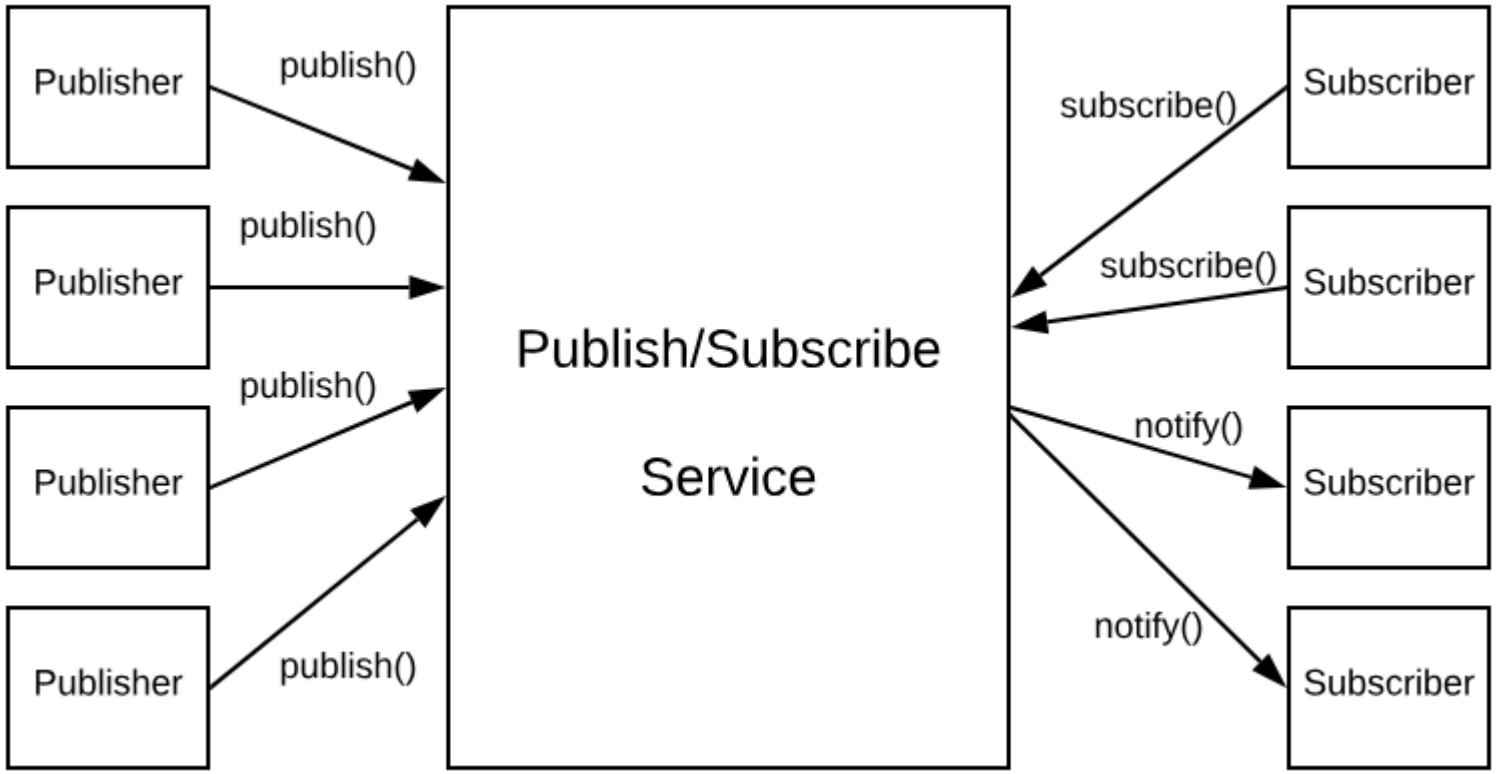
Tecnologías

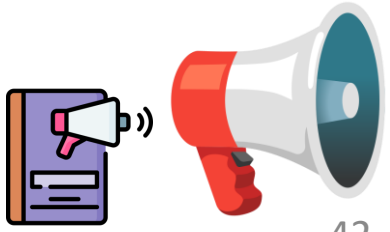
- RabbitMQ
- JMS

2. Estilos arquitectónicos – Eventos – Publicar y suscribir

Propósito

Permite a los objetos invocar servicios de otros objetos sin necesidad de conocer sus identidades.

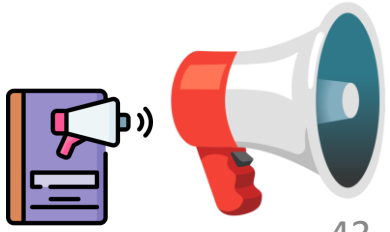




2. Estilos arquitectónicos – Eventos – Publicar y suscribir

Características

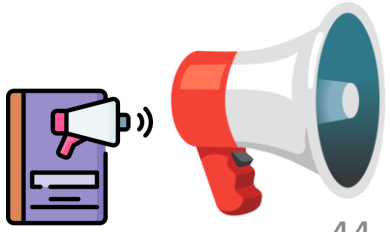
- Desacopla la comunicación.
  Emisores y receptores no interactúan directamente.
- Basada en eventos.
  Los eventos se emiten (*push*) cuando ocurren.
- Organizada en tópicos.
  - Los eventos se organizan en categorías (*topics*) .
  - Los receptores solo reciben los eventos a los que están suscritos.
- Organización del sistema en servicios.
  Los servicios pueden escalar independientemente.
- Los suscriptores reciben las actualizaciones en tiempo real.



2. Estilos arquitectónicos – Eventos – Publicar y suscribir

Utilidad

- Mantiene desacoplados los componentes del sistema.
- Los componentes no deben pasarse grandes cantidades de información entre sí (Eventos ligeros)
- Las interfaces de los componentes no están bien definidas o se esperan cambios
  - ¿Cuál es la interfaz de los componentes?
  - Se definen a nivel de los eventos
- Se quiere mantener independientes a los distintos subsistemas.
  Por ejemplo, tenemos sub-sistemas que implementen otros estilos arquitectónicos.



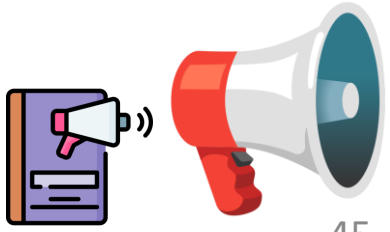
2. Estilos arquitectónicos – Eventos – Publicar y suscribir

Ejemplos de uso

- Notificaciones en tiempo real (e.g., precios de acciones, feeds de noticias)
- Servicios auxiliaries (e.g., logging, monitoring).
- IoT (e.g., streaming de datos de sensores).

Ejemplo de tecnología

- Kafka



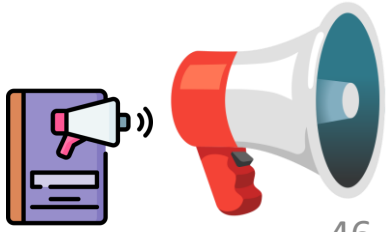
2. Estilos arquitectónicos – Eventos – Publicar y suscribir

```
# Publisher (servicio de pedidos)
from kafka import KafkaProducer
import json

producer = KafkaProducer(bootstrap_servers='localhost:9092')
order_event = {"order_id": 123,
       "customer_id": 456,
       "items": ["item1", "item2"]}
producer.send('order_topic’,
       json.dumps(order_event).encode('utf-8'))
```

```
# Subscriber (servicio de inventario)
from kafka import KafkaConsumer

consumer = KafkaConsumer('order_topic’,
       bootstrap_servers='localhost:9092')
for message in consumer:
    print("Inventory Service received:", message.value)
```



2. Estilos arquitectónicos – Eventos – Diferencias

Característica	Publicar y suscribir	Colas de mensajes
Enrutado	Broadcast a múltiples suscriptores.	Enviado a una cola específica.
Garantías de entrega	Ninguna (excepto si es en modo retenido)	Almacenado en la cola hasta la entrega.
Modelo de procesamiento	Tiempo real, event-driven.	Asíncrono, message-driven.
Casos de uso	Notificaciones, actualizaciones en tiempo real.	Procesamiento fiable de tareas.

Colas

- La entrega tiene que ser fiable.
- Los servicios procesan los mensajes en orden.

Publicar y suscribir

- Varios servicios necesitan reaccionar de manera independiente a un evento.

2. Estilos arquitectónicos – Cliente y servidor

Funcionamiento

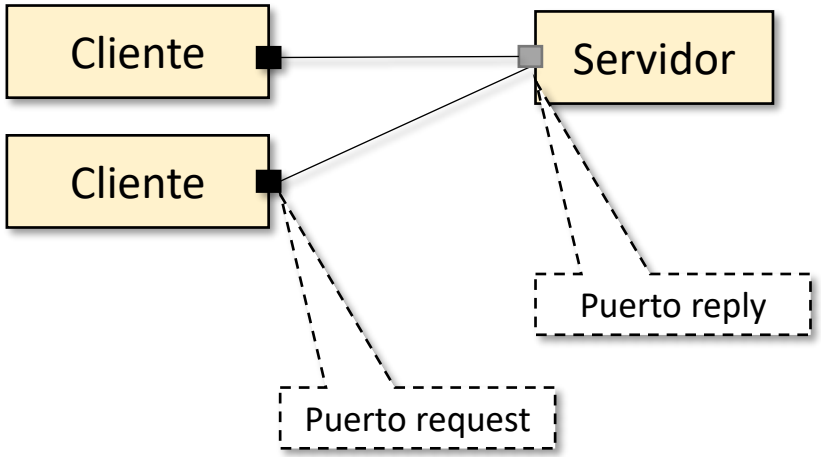
- Los **clientes** solicitan servicios a un servidor. *(requests)*
- Los **servidores** responden a las peticiones. *(reply)*

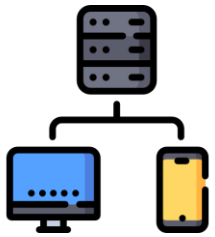
Componentes

- Cliente y servidor.

Conectores

- Request – Reply.

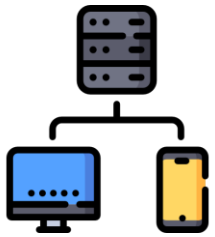




2. Estilos arquitectónicos – Cliente y servidor

Características

- La comunicación es asimétrica.
  El servidor no conoce la identidad de los clientes hasta que recibe la primera petición de un cliente.
  - El servidor no puede iniciar comunicación con un cliente
  - Los clientes saben la identidad de un servidor o cómo localizarlo (IP, DNS, …)
- Suele haber un límite en el número de conexiones simultáneas.
- La comunicación suele ser síncrona.
  Puede ser asíncrona, pero siempre iniciada por el cliente
- Comunicación sin estado (*stateless*) o con estado (*stateful*)



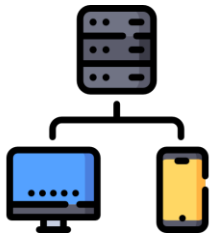
2. Estilos arquitectónicos – Cliente y servidor

Ventajas

- Mantenibilidad.
  El servidor puede actualizarse sin actualizar a los clientes.
- Evolución más sencilla.
  Siempre que no se cambie la interfaz del servidor.
- Facilidad para integrar sistemas.
  El servidor actúa como fachada de sistemas internos.
- Escalabilidad.
  Añadiendo más servidores
  (necesita un balanceador de carga)

Desventajas

- Dependencia de la red.
- Coste.
- Necesidad de organización adecuada.
  Por ejemplo: balanceador de carga para evitar que haya un único punto de fallo.

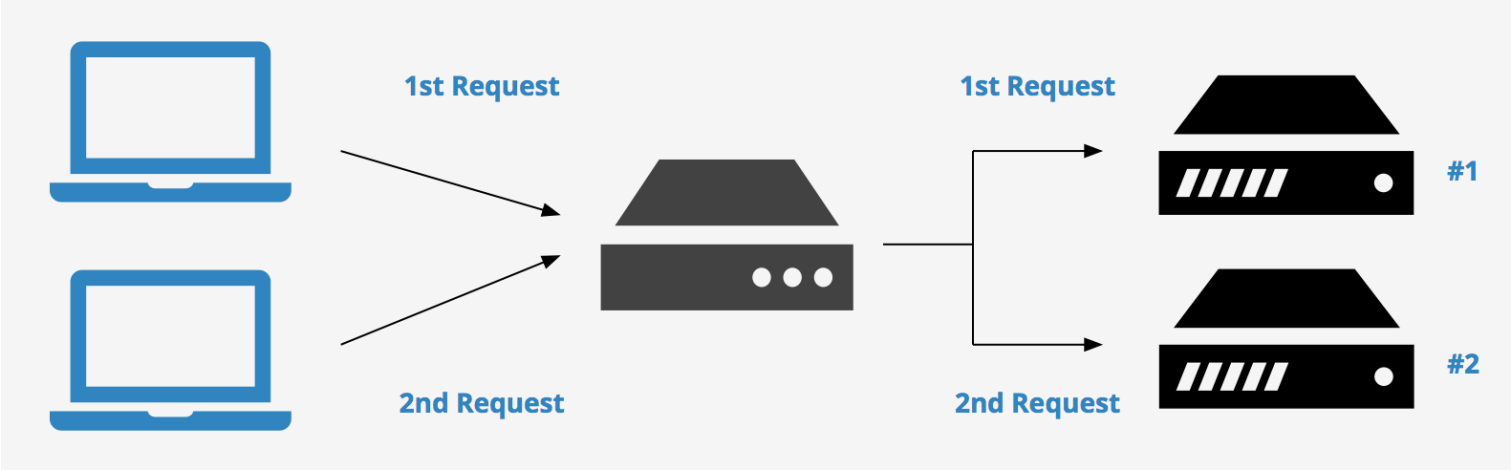


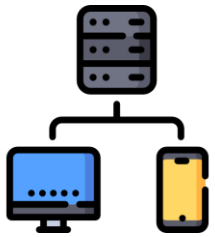
2. Estilos arquitectónicos – Cliente y servidor

Escalado horizontal

- Varios servidores con balanceador de carga.
- Reparto de peticiones.

Objetivo:
Soportar más usuarios y carga



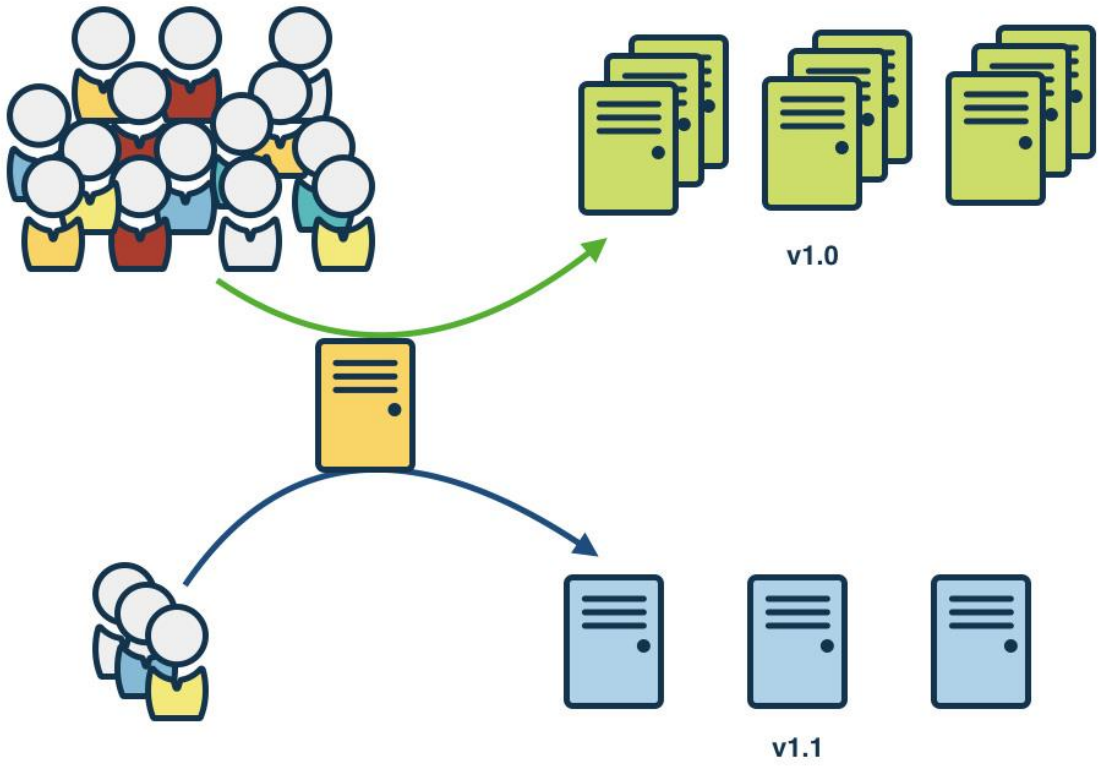


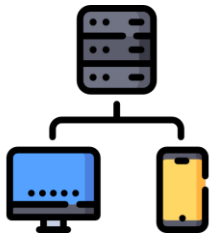
2. Estilos arquitectónicos – Cliente y servidor

Despliegue progresivo

- Podemos probar nuevas versiones en producción.
- Redirigir el tráfico.
- Comparar comportamiento.

Objetivo:
Reducir riesgos al desplegar cambios.

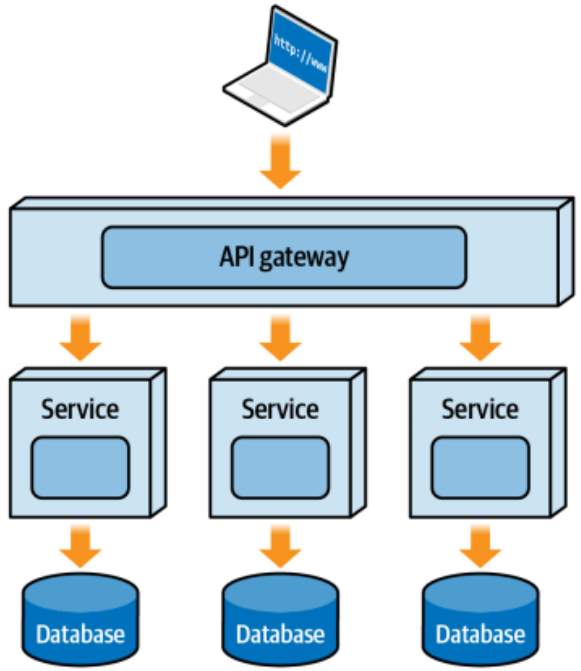




2. Estilos arquitectónicos – Microservicios

Organizar un sistema como un conjunto de servicios, que:

- Tienen un propósito único y simple (micro)
- Evolucionan de manera independiente
- Accedidos mediante una API
- Cada servicio accede a sus propios datos (*data ownership*)
- Comunicación con otros servicios para interactuar con sus datos



2. Estilos arquitectónicos – Microservicios

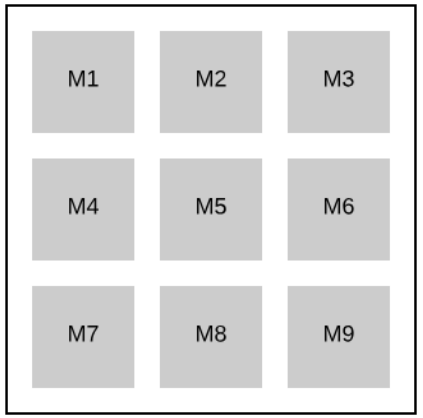
Arquitectura monolítica

- División en módulos.
- Todos se ejecutan en el mismo proceso.
- Un cambio puede afectar diversos módulos.
  - Hay que controlar cambios hechos por diversos equipos.
  - No es posible desplegar un único módulo o cambio (todo o nada)

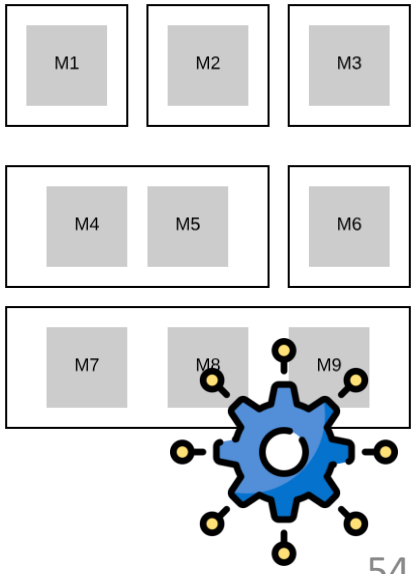
Microservicios

- Descomposición de módulos *"en tiempo de ejecución"*
- Un microservicio accede a otro a través de su API
  (ocultación de información, no datos compartidos)
- Mientras la API no cambie, los servicios pueden evolucionar.

Monolito con 9 módulos.
Todos se ejecutan en el mismo proceso.



División 6 microservicios



2. Estilos arquitectónicos – Microservicios

Ventajas

- Facilitan la organización en equipos.
  Asignar un servicio a un equipo
- Escalabilidad
  Asignación dinámica de recursos.
- Cada servicio usa tecnologías diferentes.
- Cada servicio se despliega independientemente.
- Tolerancia a fallos.
  Si falla un microservicio, no tiene por qué fallar todo el sistema.

Desventajas

- La gestión de microservicios es mucho más complicada.
- Coste de la infraestructura.
- Rendimiento más pobre debido a la sobrecarga de la comunicación.

3. Ejercicios

Vamos a poner en práctica lo que hemos estado viendo

3. Ejercicios – Juegos multijugador

UMUSoccer

Requisitos funcionales

- Sincronización en tiempo real.
- Múltiples jugadores simultáneos.
- Interacción continua.
- Evitar las trampas.

Requisitos técnicos

- Baja latencia (respuesta inmediata).
- Escalabilidad.
- Control de la autoridad.

https://www.gabrielgambetta.com/client-server-game-architecture.html

3. Ejercicios – Procesamiento de nóminas

UMUNónimas

Requisitos funcionales

- Procesar nóminas de entidades públicas.
- Basada en normativa estatal.
- Cada entidad tiene reglas propias.

Requisitos técnicos

- Variabilidad por cliente.
- Personalización.
  Cada ayuntamiento paga diferente y el cálculo del sueldo es diferente.
- Despliegue progresivo.
  Primera versión para unos pocos ayuntamientos. Progresivamente se irá ofertando a nuevos ayuntamientos.

3. Ejercicios – Viajes en coche

UMUCar

Requisitos funcionales

- Cliente solicita viaje.
- Conductores reciben notificación.
- Conduces envían propuestas.
- Cliente selecciona una propuesta.
- Actualización de la ruta en tiempo real.

Requisitos técnicos

- Comunicación en tiempo real.
- Notificaciones a múltiples usuarios.
- Baja latencia.

UMUCar

Bibliografía

- **Software Engineering – Basic Principles and Best Practices**. Ravi Sethi. Cambridge University Press. 2023.
  - Capítulos 6 y 7
- **Just Enough Software Architecture. A Risk Driven Approach**. George Fairbanks.
  - Capítulo 1 y Capítulo 14.
- **Software Engineering. A modern approach**. Marco Tulio Valente.
  - Capítulo 7
  - https://softengbook.org/chapter7